TARDIS

Le site de prediction de retard et d’informations sur la SNCF

Théo BUSIRIS
Zachary JORIOT
Raphael LEGER

Les notebooks

01

Nettoyage des données, organisation des datasets/ dataframes et visualisation des données sous forme de graphiques.

→ notebook

02

Entrainements des modèles et utilisations des données d'entraînement pour les predictions, comparaison des modèles les plus performant et choix final.

→ notebook

LOAD DATASET

In [1]:
```
import pandas as pd
import calendar
import matplotlib.pyplot as plt

df = pd.read_csv("./dataset.csv", sep=";")
df = df.drop_duplicates()
```

CLEAN SERVICES

In [2]:
```
df["Is_Service_National"] = df["Service"] == "National"
df["Is_Service_International"] = df["Service"] == "International"

df = df.drop(columns="Service")
```

CLEAN GARE NAME

In [3]:
```
cols_gare = ["Gare d'arrivée", "
df = df.dropna(subset=cols_gare)
df[cols_gare] = df[cols_gare].as
df = df[(df["Gare d'arrivée"] !=
df = df[df["Gare d'arrivée"] !=
df = df.drop_duplicates(
    subset=["Gare de départ", "G
    keep="first"
)
```

CLEAN DATES

In [4]:
```
df["Date"] = pd.to_datetime(df["
df["Date"] = df["Date"].dt.to_pe

df = df.dropna(subset=["Date"])

df["Year"] = df["Date"].dt.year
df["month"] = df["Date"].dt.mont
```

CLEAN NUMERIC COLOMN

In [5]:
```
cols_num = [
    "Retard moyen trains en reta
```

3 - Models initialisations (instance & hyperparameters list)

Now that our training and evaluation data are ready, we create a dictionary to store all our models. For each model, we'll use its name as a key, and its instance and future hyperparameters to be tested as associated values. Hyperparameters are the parameters we define before training the model, and they influence its performance. The goal will then be to determine the most efficient hyperparameters for each model.

In [42]:
```
models_init = {
    'DummyRegressor': {
        'instance': DummyRegressor(strategy='mean'),
        'param_grid': {
            'model__strategy': ['mean', 'median']
        }
    },
    'LinearRegression': {
        'instance': LinearRegression(),
        'param_grid': {}
    },
    'HuberRegressor': {
        'instance': HuberRegressor(max_iter=1000),
        'param_grid': {
            'model__epsilon': [1.2, 1.35, 1.5, 2.0],
            'model__alpha': [0.0001, 0.001, 0.01, 0.1],
            'model__max_iter': [1000, 2000]
        }
    },
    'Ridge': {
        'instance': Ridge(alpha=1.0),
        'param_grid': {
            'model__alpha': [0.01, 0.1, 1.0, 10.0, 100.0]
        }
```

7 - Get the benchmark list of all regression models

We call this function, with the list of models, and we convert the returned list into a dataframe, before sorting it by MAE, to see the overall performance ranking of our models.

In [46]:
```
models_first_benchmark = models_benchmark(models_init)

results_df = pd.DataFrame(models_first_benchmark).sort_values(by=['MAE', 'RMSE', 'R2'], ascending=[True, True, False]).reset_index(drop=True)
top_5_best_model = results_df.head(5)

print(results_df)
```

```
                     model       MAE      RMSE        R2
0       RandomForestRegressor  2.069967  3.124404  0.446141
1         ExtraTreesRegressor  2.093868  3.215510  0.413369
2   GradientBoostingRegressor  2.294042  3.273951  0.391852
3             HuberRegressor  2.343657  3.495109  0.306915
4           LinearRegression  2.384718  3.476682  0.314204
5                      Ridge  2.385338  3.472190  0.315975
6         KNeighborsRegressor  2.468311  3.673800  0.234234
7       DecisionTreeRegressor  2.500336  4.066755  0.061658
8                 ElasticNet  2.658005  3.773389  0.192155
9           DummyRegressor  3.070297  4.198303 -0.000029
10          AdaBoostRegressor  3.605732  4.427244 -0.112070
```

8 - Show performance of each models with plots

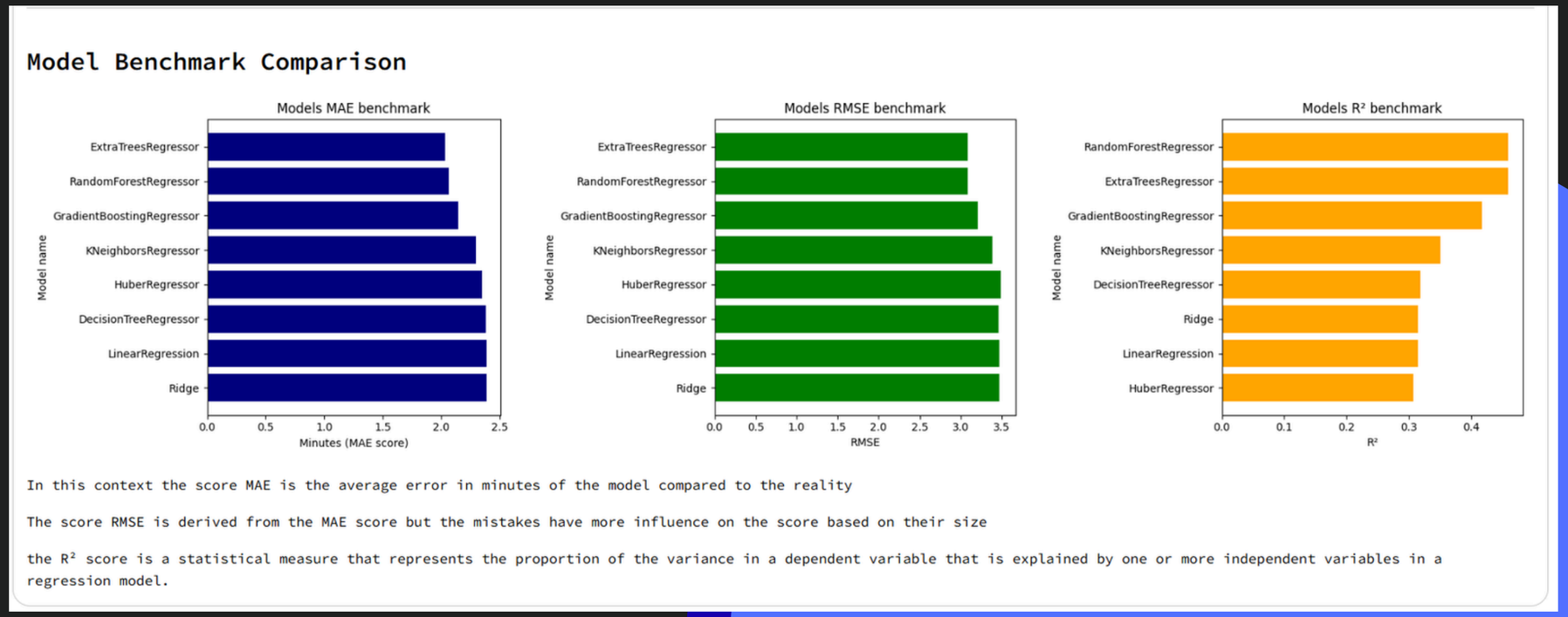
We create a Plot class to quickly generate a comparison graph of different models based on metrics collected from the test set. The constructor takes the dataframe, column names, title, x, y, and x labels, a boolean value to choose whether to display in ascending or descending order (since a lower MAE score indicates better performance, a lower R^2 score indicates worse performance), and the color as parameters. This class save as png and show the plot, includes a get_plot() method, which retrieves the graph and allows exporting it if needed.

In [47]:
```
class Plot():
    def __init__(
        self,
        df_plot: object,
```

Les modèles

Comparaison de 11 modèles différents (hyperparamètres compris)

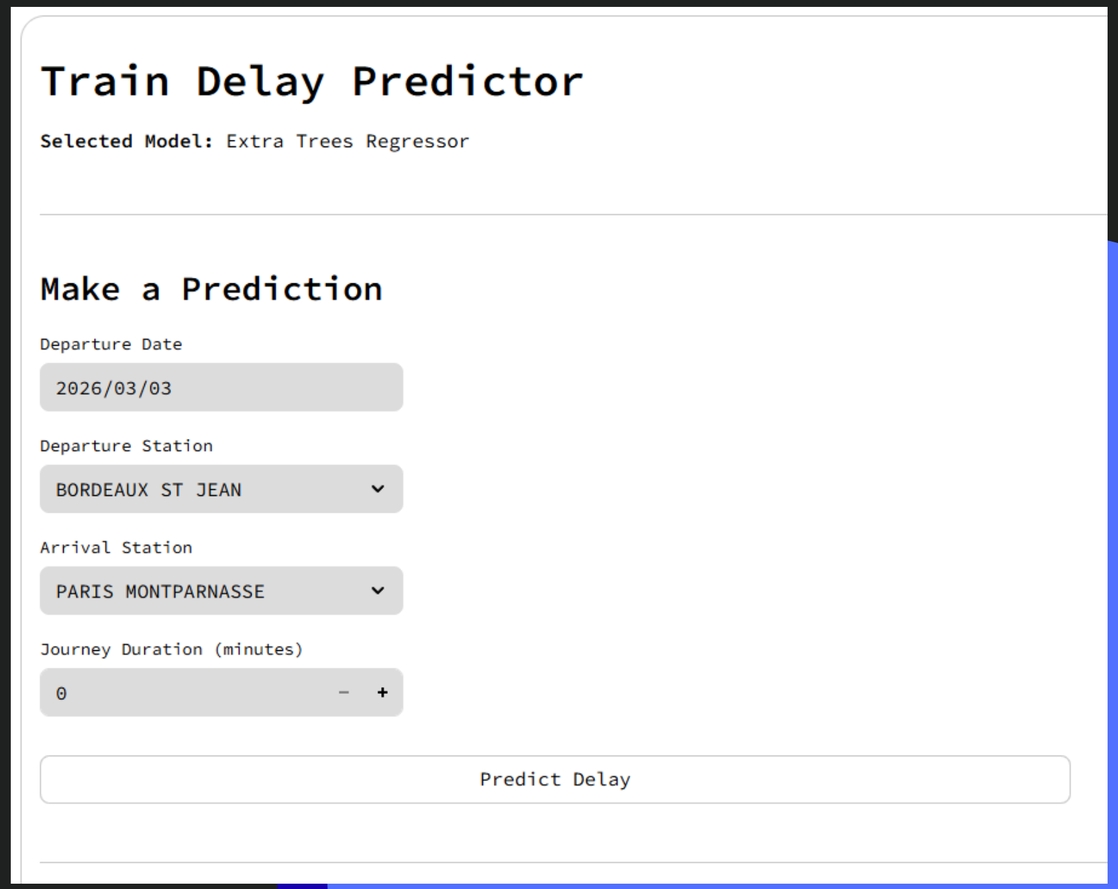
Pour le choix des modèles, nous les avons sélectionnés par grandes catégories : arbres / ensembles d'arbres, modèles linéaires, modèles basés sur la proximité et baseline. L'idée était de réaliser un benchmark avec différents types de modèles afin d'identifier celui ou ceux qui étaient les plus pertinents.



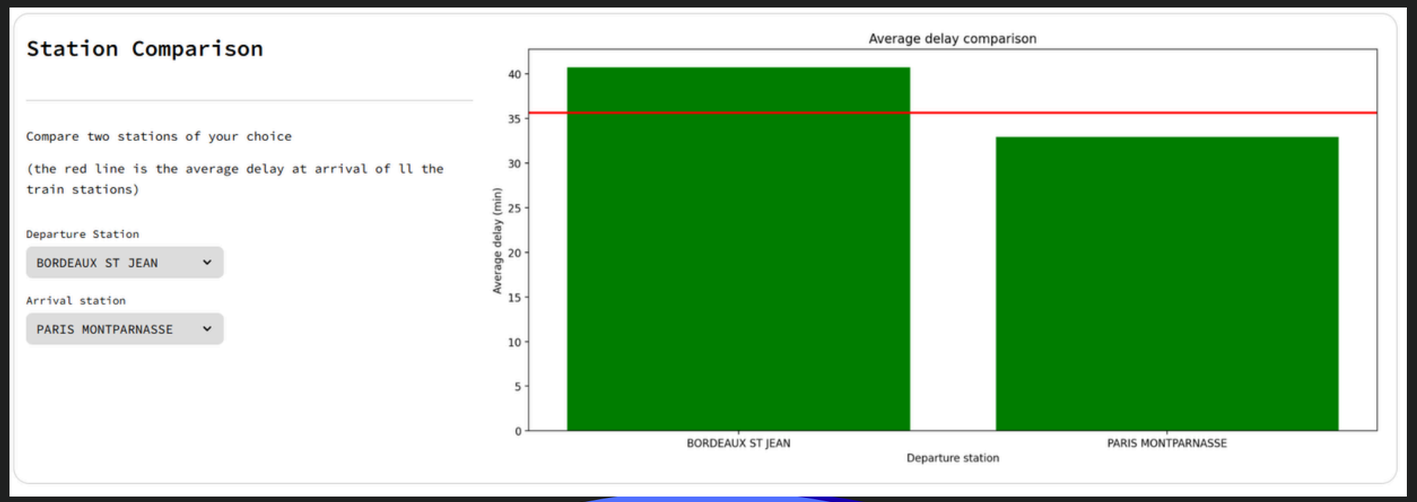
Le modèle choisi

Le modèle choisi est “Extra Trees Regressor” avec une marge d'erreur moyenne de +- 2.0 minutes (MAE) et une prédiction bien meilleure qu'un modèle linéraire classique (0.45 de R^2) avec un dataset restreint

Son fonctionnement repose sur un principe simple : il entraîne un grand nombre d’arbres de décision, avec une part d’aléatoire dans leur construction, puis il calcule la moyenne de leurs prédictions. La différence avec le "RandomForestRegressor" est qu’Extra Trees utilise des seuils de découpe choisis aléatoirement lors de la construction des arbres. On observe d’ailleurs ici que leurs performances sont très proches.



Les modules



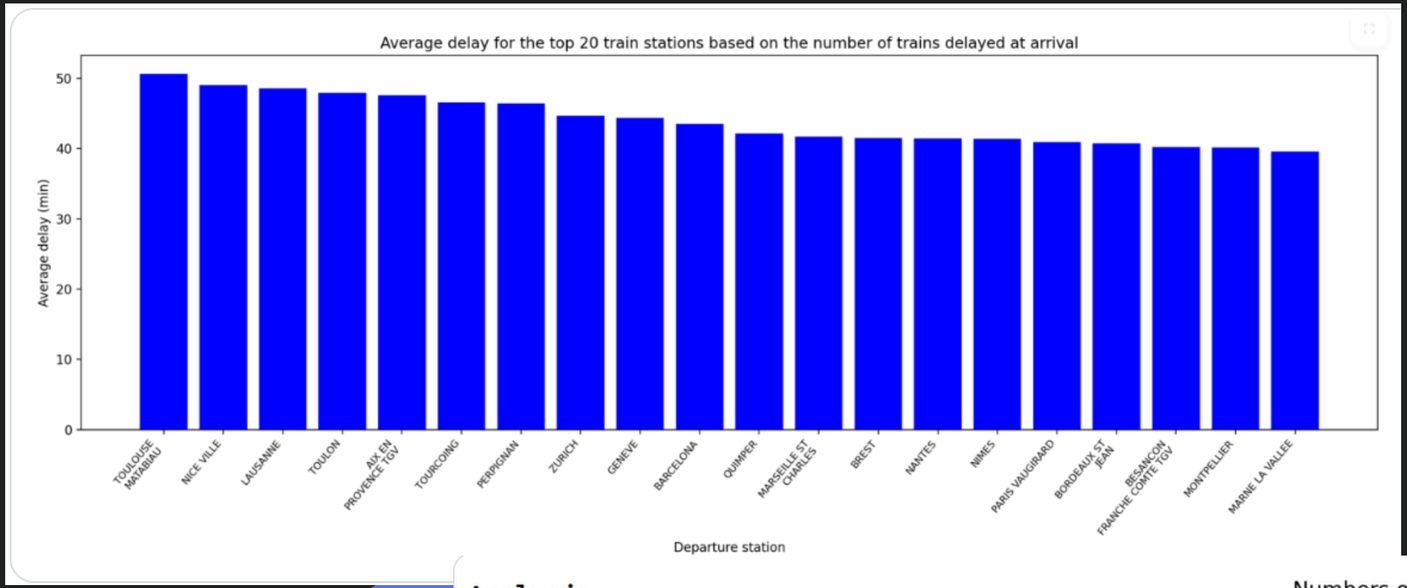
Tardis possède 2 modules distincts sur les retards des trains :

- Le premier est un module de comparaison entre deux gare au choix
- Le second est un module qui prédit les retards des futurs trains en fonction des gares d'arrivées et de départ , du temps de trajet et de la date du voyage

Les graphiques

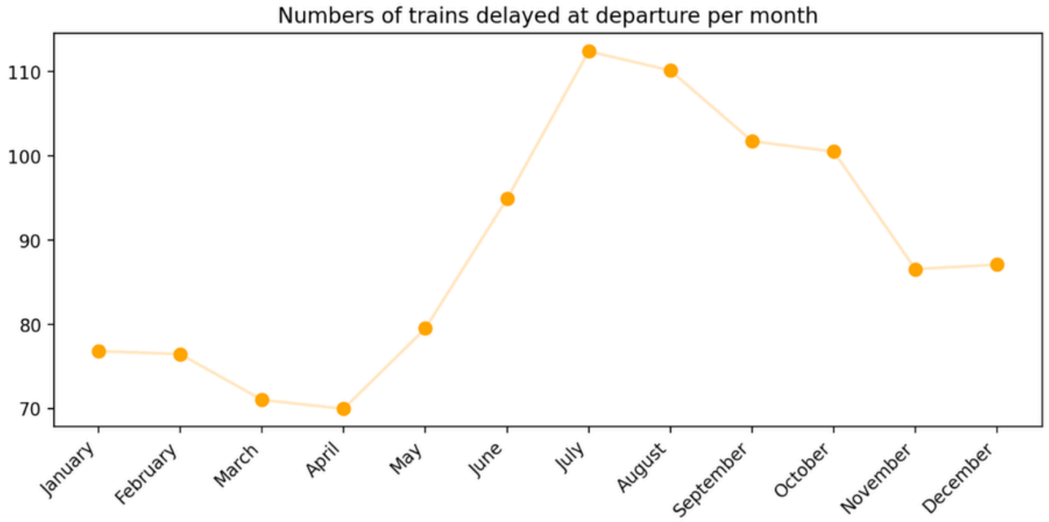
Tardis possède 2 principaux graphiques :

- Le premier est un graphique représentant les 20 villes avec le grand retard moyen a l'arrivée
- Le second est un graphique représentant le nombre de trains en retards en fonction du mois de l'année



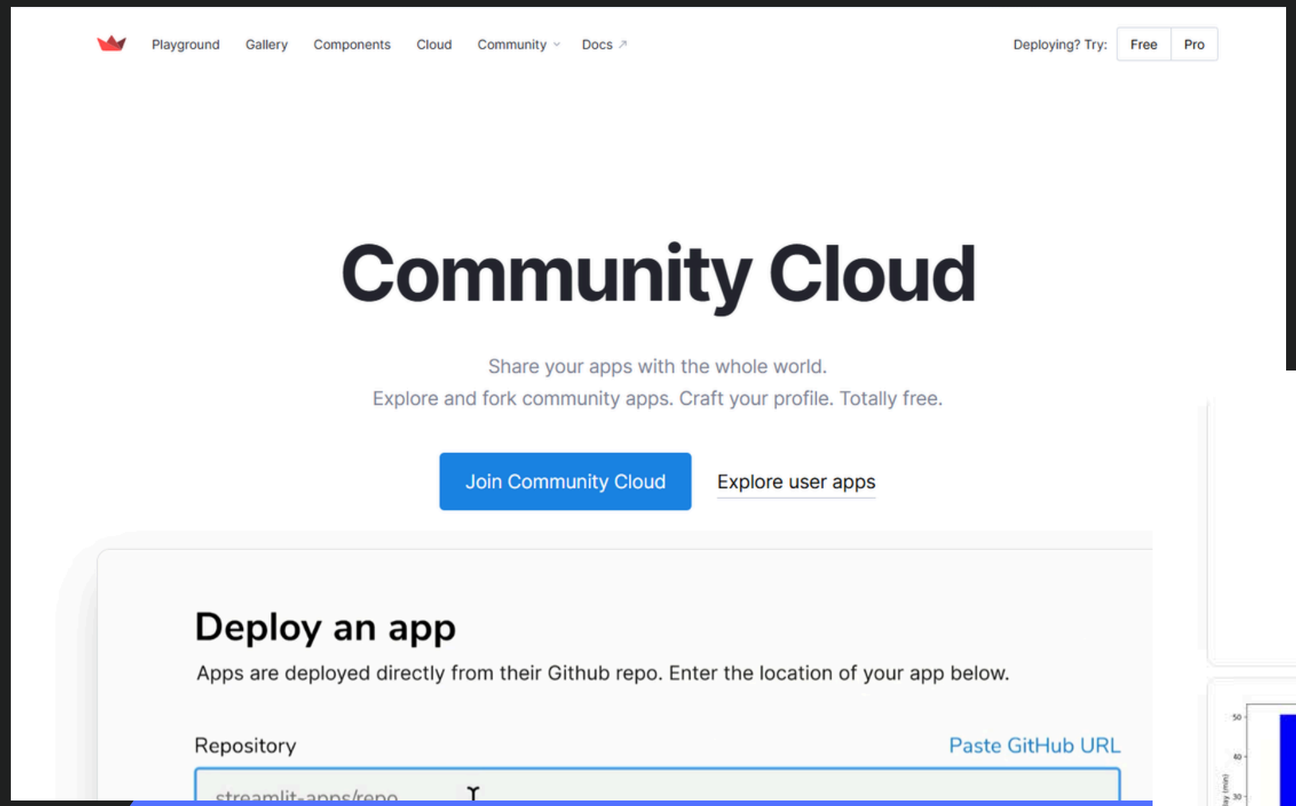
Analysis

Average delayed trains by month based on the dataset given by the SCNF on the 2018-2025 period, peaking in the summer due to the amount of trains.



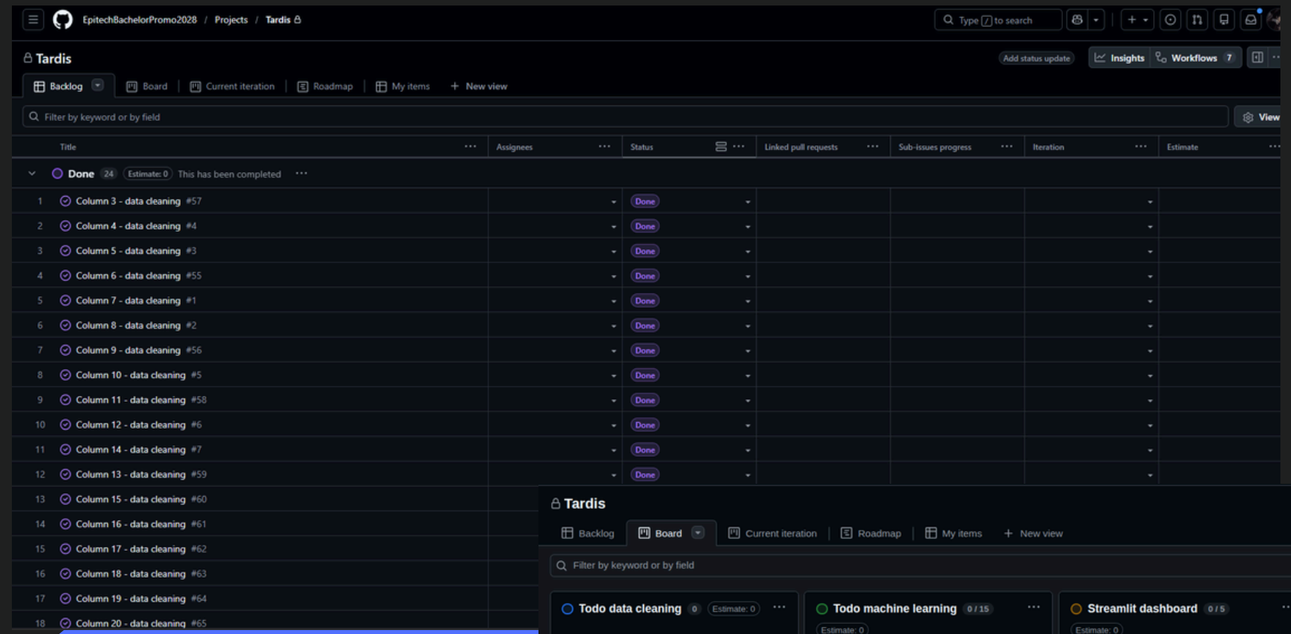
L’hébergement

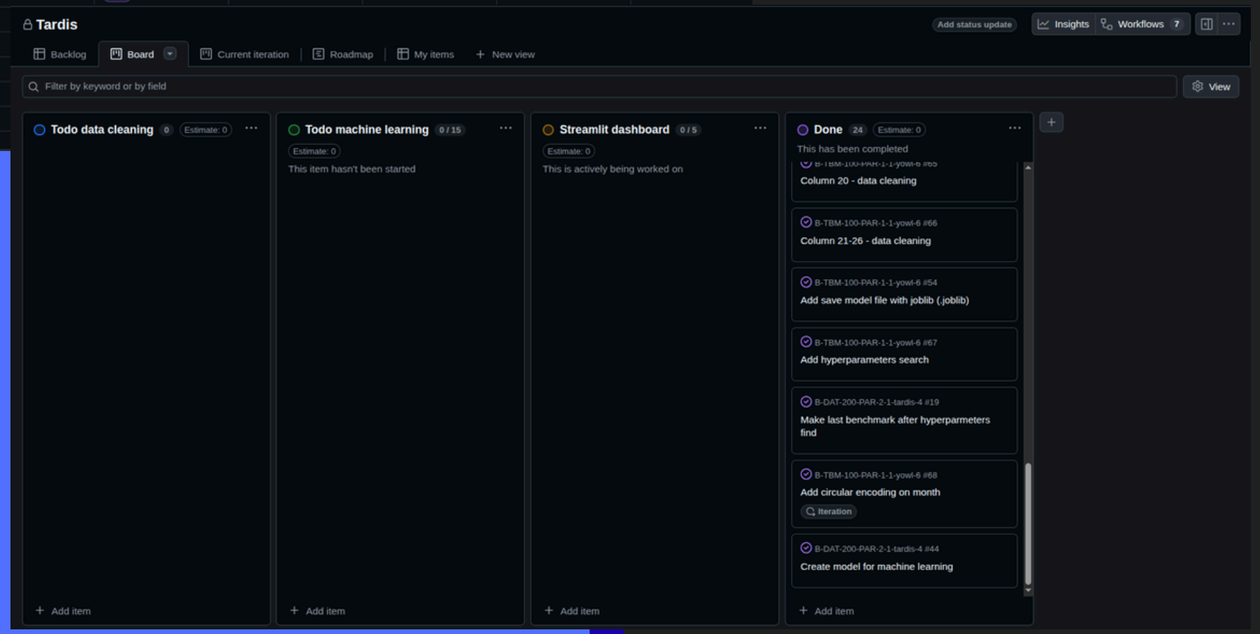
Tardis est hébergé sur l’outil Streamlit community qui permet un hébergement simple et rapide des applications développées sur Streamlit.



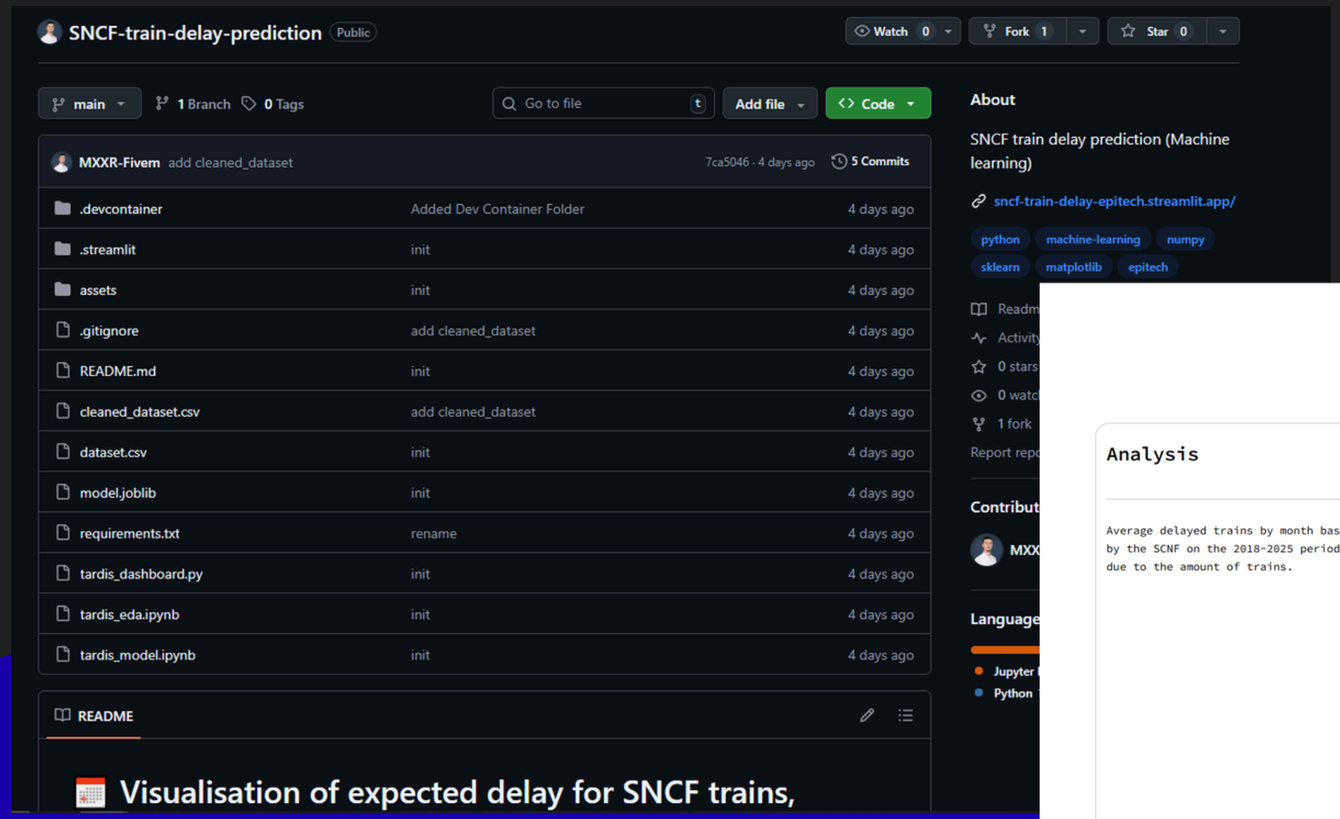
Les outils

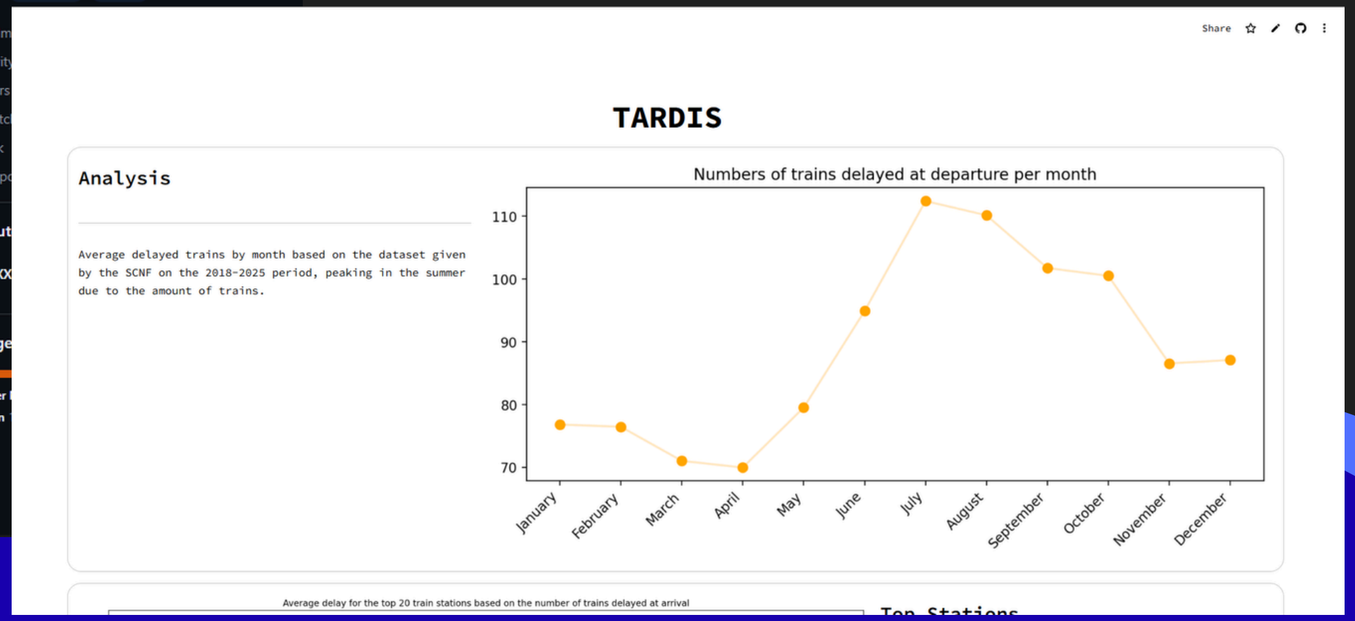
Pour le développement nous avons utilise GitHub Project





Thank You


SNCF-train-delay-prediction
Public
Watch 0
Fork 1
Star 0
main
1 Branch
0 Tags
Go to file
Add file
Code
MXXR-Fivem add cleaned_dataset
7ca5046 · 4 days ago
5 Commits
.devcontainer Added Dev Container Folder 4 days ago
.streamlit init 4 days ago
assets init 4 days ago
.gitignore add cleaned_dataset 4 days ago
README.md init 4 days ago
cleaned_dataset.csv add cleaned_dataset 4 days ago
dataset.csv init 4 days ago
model.joblib init 4 days ago
requirements.txt rename 4 days ago
tardis_dashboard.py init 4 days ago
tardis_eda.ipynb init 4 days ago
tardis_model.ipynb init 4 days ago
README
Visualisation of expected delay for SNCF trains,
About
SNCF train delay prediction (Machine learning)
sncf-train-delay-epitech.streamlit.app/
python
machine-learning
numpy
sklearn
matplotlib
epitech
1 fork
Contribut
Language
Jupyter
Python


Share
TARDIS
Analysis
Average delayed trains by month based on the dataset given by the SCNF on the 2018-2025 period, peaking in the summer due to the amount of trains.
Numbers of trains delayed at departure per month
110
100
90
80
70
January
February
March
April
May
June
July
August
September
October
November
December
Average delay for the top 20 train stations based on the number of trains delayed at arrival